

Pontificia Universidad Católica de Chile

EPG4001 Aprendizaje Supervisado

Material Complementario Adicional

MIAApSupClase7A.pdf

Jorge Luis Bazán
jlbazan@uc.cl

Librerías

Las librerías a utilizar para los procedimientos son las siguientes:

```
library(e1071)      # svm(), tune(), naiveBayes()
library(class)      # knn()
library(caret)      # confusionMatrix(), trainControl(), train()
library(ISLR2)      # Base de datos Default
library(MASS)       # lda(), qda()
library(naivebayes)  # naive_bayes(), usado por caret::train()
library(pROC)       # Curvas ROC y AUC
library(ggplot2)    # Visualización
library(tidyverse)  # Manipulación de datos
library(knitr)      # Tablas con kable()
library(themis)     # SMOTE
```

1. Base de Datos: Default

La base `Default` contiene informacion de **10,000 clientes** de tarjetas de credito. La variable respuesta es `default` (impago: **Yes / No**) en funcion de tres predictores: `balance` (saldo promedio), `income` (ingreso anual) y `student` (si es o no estudiante).

```
data(Default, package = "ISLR2")
Default <- Default %>%
  mutate(default_bin = ifelse(default == "Yes", 1, 0))
glimpse(Default)

## Rows: 10,000
## Columns: 5
## $ default      <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No~
## $ student      <fct> No, Yes, No, No, No, Yes, No, Yes, No, No, Yes, Yes, No, N~
## $ balance      <dbl> 729.5265, 817.1804, 1073.5492, 529.2506, 785.6559, 919.588~
## $ income       <dbl> 44361.625, 12106.135, 31767.139, 35704.494, 38463.496, 749~
## $ default_bin  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~

Default %>%
  count(default) %>%
  mutate(proporcion = round(n / sum(n), 3)) %>%
  kable(caption = "Distribucion de la variable respuesta Default",
        booktabs = TRUE)
```

Cuadro 1: Distribucion de la variable respuesta Default

default	n	proporcion
No	9667	0.967
Yes	333	0.033

Los datos presentan un marcado desbalance: solo el 3.3% de los clientes incurre en impago. Esto tiene efecto en las metricas de desempeno, especialmente en la sensibilidad.

```
ggplot(Default, aes(x = balance, y = income, color = default)) +
  geom_point(alpha = 0.3, size = 0.6) +
  scale_color_manual(values = c("steelblue", "tomato")) +
  labs(x = "Balance", y = "Income", color = "Default") +
  theme_minimal(base_size = 11)
```

Los clientes con impago se concentran en valores altos de `balance`.

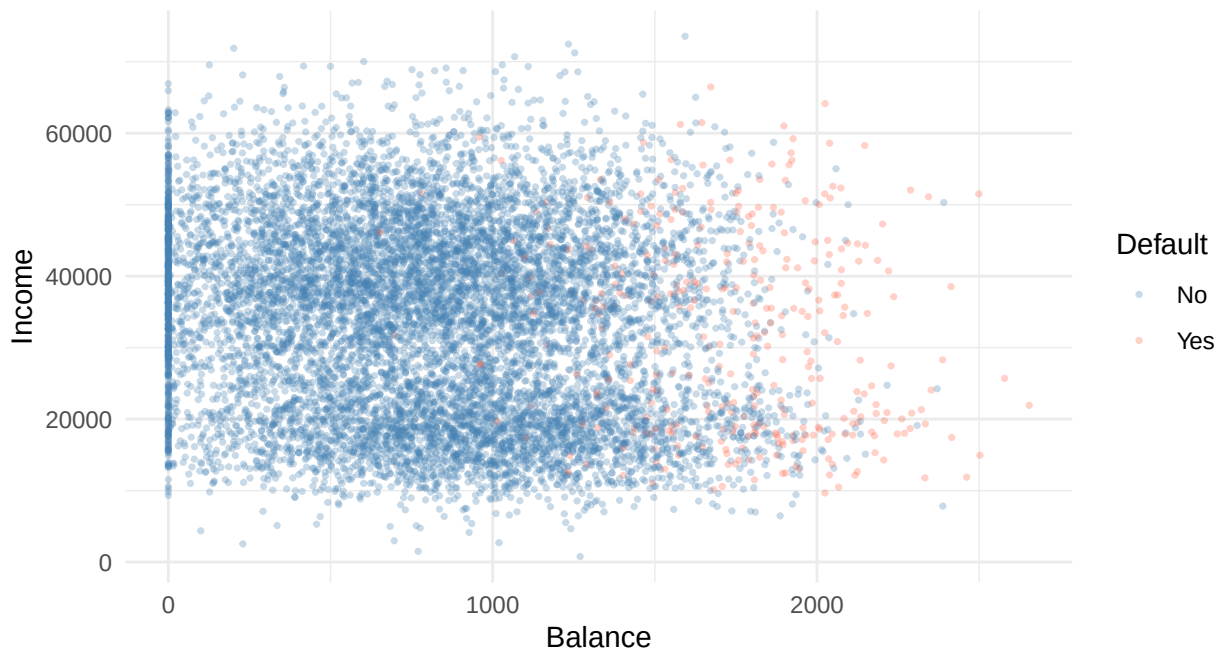


Figura 1: Balance vs Income segun Default

2. Ajuste de los Modelos de Clasificacion

Se ajustan los cuatro clasificadores de la Clase 7 sobre el conjunto completo de datos, usando **balance** e **income** como predictores. Con fines comparativos, se agregan ademas los cuatro clasificadores de la Clase 5 (Regresion Logistica, Naive Bayes, LDA y QDA), ajustados sobre los mismos predictores **balance** e **income** para que la comparacion final sea directa.

2.1. k-NN

El k-NN clasifica cada observacion por la moda de sus k vecinos mas cercanos en el espacio de predictores. Se ajusta via **caret** para obtener probabilidades de clase (necesarias para AUC):

```
set.seed(123)

# Estandarizar predictores (obligatorio para k-NN)
pre_sc <- preProcess(Default[, c("balance", "income")],
                      method = c("center", "scale"))
Default_sc <- Default
Default_sc[, c("balance_sc", "income_sc")] <-
  predict(pre_sc, Default[, c("balance", "income")])

ctrl_knn <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary)

mod_knn <- train(default ~ balance_sc + income_sc,
                 data      = Default_sc,
                 method    = "knn",
                 trControl = ctrl_knn,
```

```

metric      = "ROC",
tuneGrid    = data.frame(k = c(5, 10, 15, 20, 30)))

# Mejor k seleccionado por CV:
mod_knn$bestTune

```

```

##      k
## 5 30

```

2.2. SVM Lineal

```

ctrl_svm <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary)

```

```

set.seed(123)
mod_svm_lin <- train(default ~ balance + income,
                    data      = Default,
                    method    = "svmLinear",
                    trControl  = ctrl_svm,
                    metric     = "ROC",
                    preProcess = c("center", "scale"),
                    tuneGrid   = data.frame(C = c(0.01, 0.1, 1, 5)))

```

```

## maximum number of iterations reached 0.01320295 0.01118927maximum number of iterations reached 0.01118927

```

```

# Mejor C - SVM Lineal:
mod_svm_lin$bestTune

```

```

##      C
## 2 0.1

```

2.3. SVM Radial

```

set.seed(123)
mod_svm_rbf <- train(default ~ balance + income,
                    data      = Default,
                    method    = "svmRadial",
                    trControl  = ctrl_svm,
                    metric     = "ROC",
                    preProcess = c("center", "scale"))

```

```

# Mejores hiperparametros - SVM Radial:
mod_svm_rbf$bestTune

```

```

##      sigma C
## 3 1.226968 1

```

2.4. SVM Polinomial

```
set.seed(123)
mod_svm_poly <- train(default ~ balance + income,
                      data      = Default,
                      method    = "svmPoly",
                      trControl = ctrl_svm,
                      metric     = "ROC",
                      preprocess = c("center", "scale"),
                      tuneGrid   = expand.grid(degree = c(2, 3),
                                              scale  = 1,
                                              C       = c(0.1, 1)))

# Mejores hiperparametros - SVM Polinomial:
mod_svm_poly$bestTune
```

```
## degree scale C
## 1      2     1 0.1
```

2.5. Regresion Logistica

El modelo logístico (Clase 3) modela directamente la probabilidad posterior y sirve de referencia lineal para comparar con los clasificadores no lineales de la Clase 7:

$$\log \frac{P(G = 1 | \mathbf{x})}{P(G = 0 | \mathbf{x})} = \beta_0 + \beta_1 \text{balance} + \beta_2 \text{income}$$

```
mod_logit <- glm(default ~ balance + income,
                 data      = Default,
                 family    = binomial(link = "logit"))
summary(mod_logit)
```

```
##
## Call:
## glm(formula = default ~ balance + income, family = binomial(link = "logit"),
##      data = Default)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.154e+01  4.348e-01 -26.545  < 2e-16 ***
## balance      5.647e-03  2.274e-04  24.836  < 2e-16 ***
## income       2.081e-05  4.985e-06   4.174  2.99e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2920.6  on 9999  degrees of freedom
## Residual deviance: 1579.0  on 9997  degrees of freedom
## AIC: 1585
##
## Number of Fisher Scoring iterations: 8
```

2.6. Naive Bayes

Naive Bayes (Clase 5) estima la probabilidad posterior via teorema de Bayes asumiendo independencia condicional de los predictores dado el grupo:

$$P(G = k \mid \mathbf{x}) \propto \pi_k \prod_{j=1}^p f_{kj}(x_j)$$

```
mod_nb <- naiveBayes(default ~ balance + income,  
                      data = Default)
```

2.7. Analisis Discriminante Lineal (LDA)

LDA (Clase 5) asume densidad Gaussiana multivariada con matriz de covarianza comun Σ para todos los grupos, dando lugar a fronteras de decision lineales:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \hat{\mu}_k - \frac{1}{2} \hat{\mu}_k^\top \Sigma^{-1} \hat{\mu}_k + \log(\hat{\pi}_k)$$

```
mod_lda <- lda(default ~ balance + income,  
              data = Default)
```

2.8. Analisis Discriminante Cuadratico (QDA)

QDA (Clase 5) permite matrices de covarianza distintas por grupo (Σ_k), produciendo fronteras cuadraticas y mayor flexibilidad:

$$\delta_k(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \hat{\mu}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \hat{\mu}_k) - \frac{1}{2} \log |\Sigma_k| + \log(\hat{\pi}_k)$$

```
mod_qda <- qda(default ~ balance + income,  
              data = Default)
```

3. Metricas de Desempeno en Muestra Completa

Se obtienen las probabilidades predichas por cada modelo y se clasifican con umbral 0.5 para calcular las metricas de desempeno sobre la muestra completa.

```
y      <- Default$default
y_bin  <- Default$default_bin

# Probabilidades predichas
pi_knn  <- predict(mod_knn,      newdata = Default_sc, type = "prob")[, "Yes"]
pi_lin  <- predict(mod_svm_lin,  newdata = Default,   type = "prob")[, "Yes"]
pi_rbf  <- predict(mod_svm_rbf,  newdata = Default,   type = "prob")[, "Yes"]
pi_poly <- predict(mod_svm_poly, newdata = Default,   type = "prob")[, "Yes"]
pi_logit <- fitted(mod_logit)
pi_nb   <- predict(mod_nb,      newdata = Default, type = "raw")[, "Yes"]
pi_lda  <- predict(mod_lda,     newdata = Default)$posterior[, "Yes"]
pi_qda  <- predict(mod_qda,     newdata = Default)$posterior[, "Yes"]

# Funcion auxiliar de clasificacion con umbral 0.5
clf <- function(p) factor(ifelse(p >= 0.5, "Yes", "No"), levels = c("No", "Yes"))

y_knn  <- clf(pi_knn)
y_lin  <- clf(pi_lin)
y_rbf  <- clf(pi_rbf)
y_poly <- clf(pi_poly)
y_logit <- clf(pi_logit)
y_nb   <- clf(pi_nb)
y_lda  <- clf(pi_lda)
y_qda  <- clf(pi_qda)
```

3.1. Matrices de Confusion

```
cm_knn <- confusionMatrix(y_knn, y, positive = "Yes")

# Matriz de confusion - k-NN:
cm_knn$table

##           Reference
## Prediction   No  Yes
##           No 9629 226
##           Yes  38 107

kable(cm_knn$table,
      caption = "Matriz de Confusion - k-NN (umbral 0.5)",
      booktabs = TRUE)
```


Cuadro 2: Matriz de Confusion - k-NN (umbral 0.5)

	No	Yes
No	9629	226
Yes	38	107

```
cm_lin <- confusionMatrix(y_lin, y, positive = "Yes")

# Matriz de confusion - SVM Lineal:
cm_lin$table
```

```
##           Reference
## Prediction  No  Yes
##           No 9667 333
##           Yes   0   0
```

```
kable(cm_lin$table,
      caption = "Matriz de Confusion - SVM Lineal (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 3: Matriz de Confusion - SVM Lineal (umbral 0.5)

	No	Yes
No	9667	333
Yes	0	0

```
cm_rbf <- confusionMatrix(y_rbf, y, positive = "Yes")

# Matriz de confusion - SVM Radial:
cm_rbf$table
```

```
##           Reference
## Prediction  No  Yes
##           No 9642 248
##           Yes  25  85
```

```
kable(cm_rbf$table,
      caption = "Matriz de Confusion - SVM Radial (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 4: Matriz de Confusion - SVM Radial (umbral 0.5)

	No	Yes
No	9642	248
Yes	25	85

```
cm_poly <- confusionMatrix(y_poly, y, positive = "Yes")
```

```
# Matriz de confusion - SVM Polinomial:
```

```
cm_poly$table
```

```
##           Reference
## Prediction    No  Yes
##           No 9651 269
##           Yes  16  64
```

```
kable(cm_poly$table,
      caption = "Matriz de Confusion - SVM Polinomial (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 5: Matriz de Confusion - SVM Polinomial (umbral 0.5)

	No	Yes
No	9651	269
Yes	16	64

```
cm_logit <- confusionMatrix(y_logit, y, positive = "Yes")
```

```
# Matriz de confusion - Regresion Logistica:
```

```
cm_logit$table
```

```
##           Reference
## Prediction    No  Yes
##           No 9629 225
##           Yes  38 108
```

```
kable(cm_logit$table,
      caption = "Matriz de Confusion - Regresion Logistica (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 6: Matriz de Confusion - Regresion Logistica (umbral 0.5)

	No	Yes
No	9629	225
Yes	38	108

```
cm_nb <- confusionMatrix(y_nb, y, positive = "Yes")
```

```
# Matriz de confusion - Naive Bayes:
```

```
cm_nb$table
```

```
##           Reference
## Prediction    No  Yes
##           No 9628 242
##           Yes  39  91
```

```
kable(cm_nb$table,
      caption = "Matriz de Confusion - Naive Bayes (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 7: Matriz de Confusion - Naive Bayes (umbral 0.5)

	No	Yes
No	9628	242
Yes	39	91

```
cm_lda <- confusionMatrix(y_lda, y, positive = "Yes")
```

```
# Matriz de confusion - LDA:
cm_lda$table
```

```
##           Reference
## Prediction  No  Yes
##           No 9647 256
##           Yes  20  77
```

```
kable(cm_lda$table,
      caption = "Matriz de Confusion - LDA (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 8: Matriz de Confusion - LDA (umbral 0.5)

	No	Yes
No	9647	256
Yes	20	77

```
cm_qda <- confusionMatrix(y_qda, y, positive = "Yes")
```

```
# Matriz de confusion - QDA:
cm_qda$table
```

```
##           Reference
## Prediction  No  Yes
##           No 9637 241
##           Yes  30  92
```

```
kable(cm_qda$table,
      caption = "Matriz de Confusion - QDA (umbral 0.5)",
      booktabs = TRUE)
```

Cuadro 9: Matriz de Confusion - QDA (umbral 0.5)

	No	Yes
No	9637	241
Yes	30	92

3.2. Metricas Resumen

Se calculan Accuracy, Sensibilidad, Especificidad y F1 Score para cada metodo. El F1 Score pondera la precision y la exhaustividad:

$$F_1 = \frac{2 VP}{2 VP + FP + FN}$$

```
extraer_metricas <- function(cm, nombre) {
  VP <- cm$table[2,2]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  data.frame(
    Metodo      = nombre,
    Accuracy     = round(cm$overall["Accuracy"], 4),
    Sensibilidad = round(cm$byClass["Sensitivity"], 4),
    Especificidad = round(cm$byClass["Specificity"], 4),
    F1           = round(2*VP / (2*VP + FP + FN), 4)
  )
}

tab_metricas <- bind_rows(
  extraer_metricas(cm_knn, paste0("k-NN (k=", mod_knn$bestTune$k, ")")),
  extraer_metricas(cm_lin, "SVM Lineal"),
  extraer_metricas(cm_rbf, "SVM Radial"),
  extraer_metricas(cm_poly, "SVM Polinomial"),
  extraer_metricas(cm_logit, "Reg. Logistica"),
  extraer_metricas(cm_nb, "Naive Bayes"),
  extraer_metricas(cm_lda, "LDA"),
  extraer_metricas(cm_qda, "QDA")
)

# Tabla de metricas en muestra completa :
tab_metricas
```

```
##              Metodo Accuracy Sensibilidad Especificidad      F1
## Accuracy...1   k-NN (k=30)  0.9736      0.3213      0.9961 0.4477
## Accuracy...2   SVM Lineal   0.9667      0.0000      1.0000 0.0000
## Accuracy...3   SVM Radial   0.9727      0.2553      0.9974 0.3837
## Accuracy...4 SVM Polinomial 0.9715      0.1922      0.9983 0.3099
## Accuracy...5 Reg. Logistica 0.9737      0.3243      0.9961 0.4509
## Accuracy...6 Naive Bayes   0.9719      0.2733      0.9960 0.3931
## Accuracy...7 LDA          0.9724      0.2312      0.9979 0.3581
## Accuracy...8 QDA          0.9729      0.2763      0.9969 0.4044
```

```
kable(tab_metricas, row.names = FALSE, booktabs = TRUE,
      caption = "Metricas de desempeno en muestra completa (umbral 0.5)")
```

Cuadro 10: Metricas de desempeno en muestra completa (umbral 0.5)

Metodo	Accuracy	Sensibilidad	Especificidad	F1
k-NN (k=30)	0.9736	0.3213	0.9961	0.4477
SVM Lineal	0.9667	0.0000	1.0000	0.0000
SVM Radial	0.9727	0.2553	0.9974	0.3837
SVM Polinomial	0.9715	0.1922	0.9983	0.3099
Reg. Logistica	0.9737	0.3243	0.9961	0.4509
Naive Bayes	0.9719	0.2733	0.9960	0.3931
LDA	0.9724	0.2312	0.9979	0.3581
QDA	0.9729	0.2763	0.9969	0.4044

3.3. Log-Loss (Devianza Media)

La perdida logaritmica cuantifica que tan calibradas estan las probabilidades predichas:

$$\text{Log-Loss} = -\frac{2}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

```
log_loss <- function(y, p) {
  -2 * mean(y * log(p + 1e-15) + (1 - y) * log(1 - p + 1e-15))
}

tab_logloss <- data.frame(
  Metodo = c(paste0("k-NN (k=", mod_knn$bestTune$k, ")"),
    "SVM Lineal", "SVM Radial", "SVM Polinomial",
    "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
  LogLoss = round(c(log_loss(y_bin, pi_knn),
    log_loss(y_bin, pi_lin),
    log_loss(y_bin, pi_rbf),
    log_loss(y_bin, pi_poly),
    log_loss(y_bin, pi_logit),
    log_loss(y_bin, pi_nb),
    log_loss(y_bin, pi_lda),
    log_loss(y_bin, pi_qda)), 4)
)

# Log-Loss en muestra completa :
tab_logloss
```

```
##          Metodo LogLoss
## 1    k-NN (k=30) 0.1453
## 2      SVM Lineal 0.3670
## 3      SVM Radial 0.2244
## 4 SVM Polinomial 0.1935
## 5 Reg. Logistica 0.1579
## 6     Naive Bayes 0.1626
## 7              LDA 0.1600
## 8              QDA 0.1595
```

```
kable(tab_logloss, row.names = FALSE, booktabs = TRUE,  
      caption = "Log-Loss (Devianza Media) en muestra completa")
```

Cuadro 11: Log-Loss (Devianza Media) en muestra completa

Metodo	LogLoss
k-NN (k=30)	0.1453
SVM Lineal	0.3670
SVM Radial	0.2244
SVM Polinomial	0.1935
Reg. Logistica	0.1579
Naive Bayes	0.1626
LDA	0.1600
QDA	0.1595

Valores menores de Log-Loss indican probabilidades predichas mas calibradas.

4. Curvas ROC Comparativas

Las curvas ROC permiten evaluar el desempeño de cada clasificador a todos los umbrales posibles. El AUC resume en un unico valor la capacidad discriminante.

```
roc_knn <- roc(y_bin, pi_knn, quiet = TRUE)
roc_lin <- roc(y_bin, pi_lin, quiet = TRUE)
roc_rbf <- roc(y_bin, pi_rbf, quiet = TRUE)
roc_poly <- roc(y_bin, pi_poly, quiet = TRUE)
roc_logit <- roc(y_bin, pi_logit, quiet = TRUE)
roc_nb <- roc(y_bin, pi_nb, quiet = TRUE)
roc_lda <- roc(y_bin, pi_lda, quiet = TRUE)
roc_qda <- roc(y_bin, pi_qda, quiet = TRUE)

plot(roc_knn, col = "steelblue", lwd = 2,
     main = "Curvas ROC -- Clasificadores sobre Default")
plot(roc_lin, col = "tomato", lwd = 2, add = TRUE)
plot(roc_rbf, col = "darkgreen", lwd = 2, add = TRUE)
plot(roc_poly, col = "purple", lwd = 2, add = TRUE)
plot(roc_logit, col = "orange", lwd = 2, add = TRUE)
plot(roc_nb, col = "black", lwd = 2, add = TRUE)
plot(roc_lda, col = "deeppink", lwd = 2, add = TRUE)
plot(roc_qda, col = "brown", lwd = 2, add = TRUE)
abline(a = 0, b = 1, lty = 2, col = "gray60")

legend("bottomright",
      legend = c(
        paste0("k-NN (k=", mod_knn$bestTune$k, ") (AUC = ",
              round(pROC::auc(roc_knn), 3), ")"),
        paste0("SVM Lineal (AUC = ", round(pROC::auc(roc_lin), 3), ")"),
        paste0("SVM Radial (AUC = ", round(pROC::auc(roc_rbf), 3), ")"),
        paste0("SVM Polinomial (AUC = ", round(pROC::auc(roc_poly), 3), ")"),
        paste0("Reg. Logistica (AUC = ", round(pROC::auc(roc_logit), 3), ")"),
        paste0("Naive Bayes (AUC = ", round(pROC::auc(roc_nb), 3), ")"),
        paste0("LDA (AUC = ", round(pROC::auc(roc_lda), 3), ")"),
        paste0("QDA (AUC = ", round(pROC::auc(roc_qda), 3), ")")
      ),
      col = c("steelblue", "tomato", "darkgreen", "purple",
              "orange", "black", "deeppink", "brown"),
      lwd = 2, cex = 0.72, ncol = 2)
```

```
tab_auc <- data.frame(
  Metodo = c(paste0("k-NN (k=", mod_knn$bestTune$k, ")"),
             "SVM Lineal", "SVM Radial", "SVM Polinomial",
             "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
  AUC = round(c(pROC::auc(roc_knn), pROC::auc(roc_lin),
                pROC::auc(roc_rbf), pROC::auc(roc_poly),
                pROC::auc(roc_logit), pROC::auc(roc_nb),
                pROC::auc(roc_lda), pROC::auc(roc_qda)), 4)
)

# AUC por clasificador :
tab_auc
```

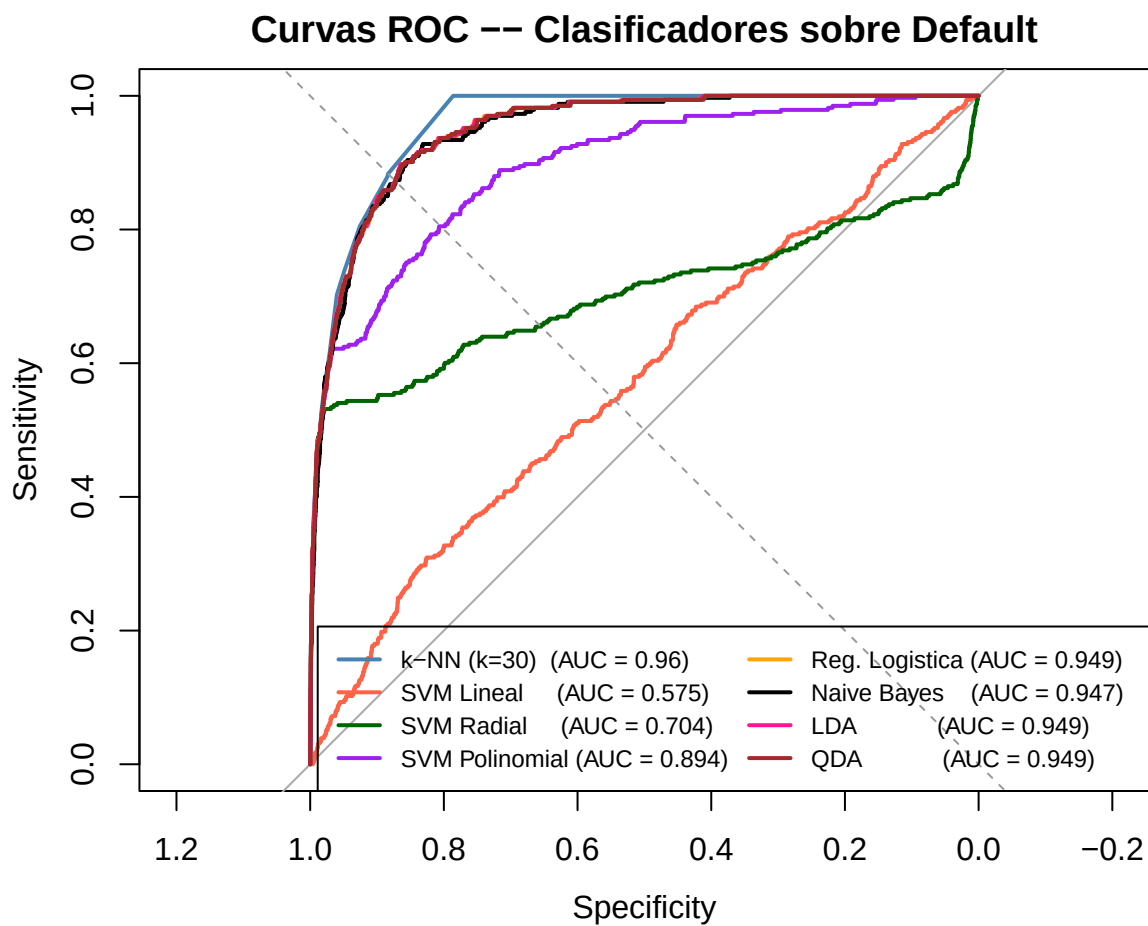


Figura 2: Curvas ROC - Comparacion de clasificadores


```
##           Metodo    AUC
## 1    k-NN (k=30) 0.9600
## 2      SVM Lineal 0.5750
## 3      SVM Radial 0.7044
## 4 SVM Polinomial 0.8941
## 5 Reg. Logistica 0.9491
## 6   Naive Bayes 0.9468
## 7           LDA 0.9491
## 8           QDA 0.9489
```

```
kable(tab_auc, row.names = FALSE, booktabs = TRUE,
      caption = "AUC por clasificador")
```

Cuadro 12: AUC por clasificador

Metodo	AUC
k-NN (k=30)	0.9600
SVM Lineal	0.5750
SVM Radial	0.7044
SVM Polinomial	0.8941
Reg. Logistica	0.9491
Naive Bayes	0.9468
LDA	0.9491
QDA	0.9489

5. Validacion Cruzada

Se estima el error de prediccion fuera de muestra mediante tres metodos de validacion cruzada, aplicados a los cuatro clasificadores de la Clase 7.

5.1. Hold-Out 70/30

Se divide aleatoriamente el conjunto en 70 % entrenamiento y 30 % prueba.

```
set.seed(123)
n <- nrow(Default)
idx <- sample(1:n, size = floor(0.7 * n))
train_d <- Default[idx, ]
test_d <- Default[-idx, ]

# Tambien version escalada para k-NN
pre_sc_ho <- preProcess(train_d[, c("balance", "income")],
                        method = c("center", "scale"))

train_sc <- train_d
test_sc <- test_d
train_sc[, c("balance_sc", "income_sc")] <-
  predict(pre_sc_ho, train_d[, c("balance", "income")])
test_sc[, c("balance_sc", "income_sc")] <-
  predict(pre_sc_ho, test_d[, c("balance", "income")])

# k-NN
set.seed(123)
mod_h_knn <- train(default ~ balance_sc + income_sc,
                  data      = train_sc,
                  method    = "knn",
                  trControl = trainControl(method = "cv", number = 5,
                                           classProbs = TRUE,
                                           summaryFunction = twoClassSummary),
                  metric     = "ROC",
                  tuneGrid   = data.frame(k = c(5, 10, 15, 20, 30)))
pi_h_knn <- predict(mod_h_knn, newdata = test_sc, type = "prob")[, "Yes"]

# SVM Lineal
set.seed(123)
mod_h_lin <- train(default ~ balance + income,
                  data      = train_d,
                  method    = "svmLinear",
                  trControl = trainControl(method = "cv", number = 5,
                                           classProbs = TRUE,
                                           summaryFunction = twoClassSummary),
                  metric     = "ROC",
                  preprocess = c("center", "scale"),
                  tuneGrid   = data.frame(C = c(0.01, 0.1, 1, 5)))
```

```
## maximum number of iterations reached 0.01606361 0.01296044maximum number of iterations reached 0.027
```

```

pi_h_lin <- predict(mod_h_lin, newdata = test_d, type = "prob")[, "Yes"]

# SVM Radial
set.seed(123)
mod_h_rbf <- train(default ~ balance + income,
  data      = train_d,
  method    = "svmRadial",
  trControl = trainControl(method = "cv", number = 5,
    classProbs = TRUE,
    summaryFunction = twoClassSummary),
  metric     = "ROC",
  preProcess = c("center", "scale"))
pi_h_rbf <- predict(mod_h_rbf, newdata = test_d, type = "prob")[, "Yes"]

# SVM Polinomial
set.seed(123)
mod_h_poly <- train(default ~ balance + income,
  data      = train_d,
  method    = "svmPoly",
  trControl = trainControl(method = "cv", number = 5,
    classProbs = TRUE,
    summaryFunction = twoClassSummary),
  metric     = "ROC",
  preProcess = c("center", "scale"),
  tuneGrid   = expand.grid(degree = c(2,3), scale = 1,
    C = c(0.1, 1)))
pi_h_poly <- predict(mod_h_poly, newdata = test_d, type = "prob")[, "Yes"]

# Regresion Logistica
mod_h_logit <- glm(default ~ balance + income,
  data = train_d, family = binomial())
pi_h_logit <- predict(mod_h_logit, newdata = test_d, type = "response")

# Naive Bayes
mod_h_nb <- naiveBayes(default ~ balance + income, data = train_d)
pi_h_nb <- predict(mod_h_nb, newdata = test_d, type = "raw")[, "Yes"]

# LDA
mod_h_lda <- lda(default ~ balance + income, data = train_d)
pi_h_lda <- predict(mod_h_lda, newdata = test_d)$posterior[, "Yes"]

# QDA
mod_h_qda <- qda(default ~ balance + income, data = train_d)
pi_h_qda <- predict(mod_h_qda, newdata = test_d)$posterior[, "Yes"]

```

```

y_test      <- test_d$default
y_test_bin  <- test_d$default_bin

metricas_holdout <- function(pi_hat, y_obs, y_bin, nombre) {
  y_pred <- clf(pi_hat)
  cm      <- confusionMatrix(y_pred, y_obs, positive = "Yes")
  VP <- cm$table[2,2]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  data.frame(

```

```

    Metodo      = nombre,
    Accuracy     = round(cm$overall["Accuracy"], 4),
    Sensibilidad = round(cm$byClass["Sensitivity"], 4),
    Especificidad = round(cm$byClass["Specificity"], 4),
    F1           = round(2*VP/(2*VP+FP+FN), 4),
    AUC          = round(as.numeric(
      pROC::auc(pROC::roc(y_bin, pi_hat, quiet=TRUE))), 4),
    LogLoss      = round(log_loss(y_bin, pi_hat), 4)
  )
}

tab_holdout <- bind_rows(
  metricas_holdout(pi_h_knn, y_test, y_test_bin,
    paste0("k-NN (k=", mod_h_knn$bestTune$k, ")")),
  metricas_holdout(pi_h_lin, y_test, y_test_bin, "SVM Lineal"),
  metricas_holdout(pi_h_rbf, y_test, y_test_bin, "SVM Radial"),
  metricas_holdout(pi_h_poly, y_test, y_test_bin, "SVM Polinomial"),
  metricas_holdout(pi_h_logit, y_test, y_test_bin, "Reg. Logistica"),
  metricas_holdout(pi_h_nb, y_test, y_test_bin, "Naive Bayes"),
  metricas_holdout(pi_h_lda, y_test, y_test_bin, "LDA"),
  metricas_holdout(pi_h_qda, y_test, y_test_bin, "QDA")
)

# Metricas Hold-Out 70/30 :
tab_holdout

```

```

##              Metodo Accuracy Sensibilidad Especificidad      F1      AUC
## Accuracy...1    k-NN (k=30)  0.9730          0.29          0.9966 0.4173 0.8961
## Accuracy...2    SVM Lineal   0.9667          0.00          1.0000 0.0000 0.9471
## Accuracy...3    SVM Radial   0.9720          0.24          0.9972 0.3636 0.6926
## Accuracy...4 SVM Polinomial  0.9687          0.09          0.9990 0.1607 0.8951
## Accuracy...5 Reg. Logistica  0.9737          0.31          0.9966 0.4397 0.9473
## Accuracy...6 Naive Bayes    0.9707          0.27          0.9948 0.3803 0.9455
## Accuracy...7      LDA       0.9713          0.22          0.9972 0.3385 0.9472
## Accuracy...8      QDA       0.9720          0.25          0.9969 0.3731 0.9471
##              LogLoss
## Accuracy...1  0.4244
## Accuracy...2  0.2898
## Accuracy...3  0.2380
## Accuracy...4  0.2051
## Accuracy...5  0.1603
## Accuracy...6  0.1637
## Accuracy...7  0.1618
## Accuracy...8  0.1617

```

```

kable(tab_holdout, row.names = FALSE, booktabs = TRUE,
  caption = "Metricas Hold-Out 70/30 - conjunto de prueba (30\\%)")

```

Cuadro 13: Metricas Hold-Out 70/30 - conjunto de prueba (30%)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC	LogLoss
k-NN (k=30)	0.9730	0.29	0.9966	0.4173	0.8961	0.4244
SVM Lineal	0.9667	0.00	1.0000	0.0000	0.9471	0.2898
SVM Radial	0.9720	0.24	0.9972	0.3636	0.6926	0.2380
SVM Polinomial	0.9687	0.09	0.9990	0.1607	0.8951	0.2051
Reg. Logistica	0.9737	0.31	0.9966	0.4397	0.9473	0.1603
Naive Bayes	0.9707	0.27	0.9948	0.3803	0.9455	0.1637
LDA	0.9713	0.22	0.9972	0.3385	0.9472	0.1618
QDA	0.9720	0.25	0.9969	0.3731	0.9471	0.1617

5.2. Validacion Cruzada K-Fold (K = 10)

Se emplea `caret` con $K = 10$ pliegues y una funcion de resumen personalizada que incluye AUC, Sensibilidad, Especificidad y Log-Loss:

```
custom_summary <- function(data, lev = NULL, model = NULL) {
  tcs <- twoClassSummary(data, lev, model)
  p <- data$Yes
  y_kf <- as.numeric(data$obs == "Yes")
  mdev <- -2 * mean(y_kf * log(p + 1e-15) + (1 - y_kf) * log(1 - p + 1e-15))
  c(tcs, LogLoss = mdev)
}
```

```
ctrl_kf <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = custom_summary,
  savePredictions = "final"
)
```

```
set.seed(123)
kf_knn <- train(default ~ balance_sc + income_sc,
  data = Default_sc,
  method = "knn",
  trControl = ctrl_kf,
  metric = "ROC",
  tuneGrid = data.frame(k = c(5, 10, 15, 20, 30)))
```

```
best_knn <- kf_knn$results[which.max(kf_knn$results$ROC), ]
cat("K-Fold (k-NN) -- AUC:", round(best_knn$ROC, 4), "\n")
```

```
## K-Fold (k-NN) -- AUC: 0.9034
```

```
cat("K-Fold (k-NN) -- Sens:", round(best_knn$Sens, 4), "\n")
```

```
## K-Fold (k-NN) -- Sens: 0.9966
```

```
cat("K-Fold (k-NN) -- Spec:", round(best_knn$Spec, 4), "\n")
```

```
## K-Fold (k-NN) -- Spec: 0.2737
```

```
cat("K-Fold (k-NN) -- LogLoss:", round(best_knn$LogLoss, 4), "\n")
```

```
## K-Fold (k-NN) -- LogLoss: 0.3805
```

```
set.seed(123)
kf_lin <- train(default ~ balance + income,
               data      = Default,
               method    = "svmLinear",
               trControl = ctrl_kf,
               metric     = "ROC",
               preProcess = c("center", "scale"),
               tuneGrid  = data.frame(C = c(0.01, 0.1, 1, 5)))
```

```
## maximum number of iterations reached 0.005060392 0.00464075maximum number of iterations reached 0.00
```

```
best_lin <- kf_lin$results[which.max(kf_lin$results$ROC), ]
cat("K-Fold (SVM Lineal) -- AUC:", round(best_lin$ROC, 4), "\n")
```

```
## K-Fold (SVM Lineal) -- AUC: 0.817
```

```
cat("K-Fold (SVM Lineal) -- Sens:", round(best_lin$Sens, 4), "\n")
```

```
## K-Fold (SVM Lineal) -- Sens: 1
```

```
cat("K-Fold (SVM Lineal) -- Spec:", round(best_lin$Spec, 4), "\n")
```

```
## K-Fold (SVM Lineal) -- Spec: 0.0152
```

```
cat("K-Fold (SVM Lineal) -- LogLoss:", round(best_lin$LogLoss, 4), "\n")
```

```
## K-Fold (SVM Lineal) -- LogLoss: 0.268
```

```
set.seed(123)
kf_rbf <- train(default ~ balance + income,
               data      = Default,
               method    = "svmRadial",
               trControl = ctrl_kf,
               metric     = "ROC",
               preProcess = c("center", "scale"))

cat("K-Fold (SVM Radial) -- AUC:", round(kf_rbf$results$ROC[1], 4), "\n")
```

```
## K-Fold (SVM Radial) -- AUC: 0.6441
```

```
cat("K-Fold (SVM Radial) -- Sens:", round(kf_rbf$results$Sens[1], 4), "\n")
```

```
## K-Fold (SVM Radial) -- Sens: 0.9973
```

```
cat("K-Fold (SVM Radial) -- Spec:", round(kf_rbf$results$Spec[1], 4), "\n")
```

```
## K-Fold (SVM Radial) -- Spec: 0.2496
```

```
cat("K-Fold (SVM Radial) -- LogLoss:", round(kf_rbf$results$LogLoss[1], 4), "\n")
```

```
## K-Fold (SVM Radial) -- LogLoss: 0.2269
```

```
set.seed(123)
kf_poly <- train(default ~ balance + income,
  data      = Default,
  method    = "svmPoly",
  trControl = ctrl_kf,
  metric    = "ROC",
  preProcess = c("center", "scale"),
  tuneGrid  = expand.grid(degree = c(2,3), scale = 1,
    C = c(0.1, 1)))
```

```
## line search fails -3.674409 -0.7954694 1.690751e-05 -1.173045e-05 -2.028411e-07 -1.540096e-07 -1.622
```

```
best_poly <- kf_poly$results[which.max(kf_poly$results$ROC), ]
cat("K-Fold (SVM Poly) -- AUC:", round(best_poly$ROC, 4), "\n")
```

```
## K-Fold (SVM Poly) -- AUC: 0.8944
```

```
cat("K-Fold (SVM Poly) -- Sens:", round(best_poly$Sens, 4), "\n")
```

```
## K-Fold (SVM Poly) -- Sens: 0.9976
```

```
cat("K-Fold (SVM Poly) -- Spec:", round(best_poly$Spec, 4), "\n")
```

```
## K-Fold (SVM Poly) -- Spec: 0.2501
```

```
cat("K-Fold (SVM Poly) -- LogLoss:", round(best_poly$LogLoss, 4), "\n")
```

```
## K-Fold (SVM Poly) -- LogLoss: 0.1941
```

```
set.seed(123)
kf_logit <- train(default ~ balance + income,
  data      = Default,
  method    = "glm",
  family    = binomial(),
  trControl = ctrl_kf,
  metric    = "ROC")

cat("K-Fold (Reg. Logistica) -- AUC:", round(kf_logit$results$ROC, 4), "\n")
```

```
## K-Fold (Reg. Logistica) -- AUC: 0.9486
```

```
cat("K-Fold (Reg. Logistica) -- Sens:", round(kf_logit$results$Sens, 4), "\n")
```

```
## K-Fold (Reg. Logistica) -- Sens: 0.9961
```

```
cat("K-Fold (Reg. Logistica) -- Spec:", round(kf_logit$results$Spec, 4), "\n")
```

```
## K-Fold (Reg. Logistica) -- Spec: 0.3247
```

```
cat("K-Fold (Reg. Logistica) -- LogLoss:", round(kf_logit$results$LogLoss, 4), "\n")
```

```
## K-Fold (Reg. Logistica) -- LogLoss: 0.1585
```

```
set.seed(123)
kf_nb <- train(default ~ balance + income,
               data      = Default,
               method    = "naive_bayes",
               trControl = ctrl_kf,
               metric    = "ROC")
```

```
best_nb_kf <- kf_nb$results[which.max(kf_nb$results$ROC), ]
cat("K-Fold (Naive Bayes) -- AUC:", round(best_nb_kf$ROC, 4), "\n")
```

```
## K-Fold (Naive Bayes) -- AUC: 0.9466
```

```
cat("K-Fold (Naive Bayes) -- Sens:", round(best_nb_kf$Sens, 4), "\n")
```

```
## K-Fold (Naive Bayes) -- Sens: 0.9961
```

```
cat("K-Fold (Naive Bayes) -- Spec:", round(best_nb_kf$Spec, 4), "\n")
```

```
## K-Fold (Naive Bayes) -- Spec: 0.2708
```

```
cat("K-Fold (Naive Bayes) -- LogLoss:", round(best_nb_kf$LogLoss, 4), "\n")
```

```
## K-Fold (Naive Bayes) -- LogLoss: 0.1632
```

```
set.seed(123)
kf_lda <- train(default ~ balance + income,
               data      = Default,
               method    = "lda",
               trControl = ctrl_kf,
               metric    = "ROC")
```

```
cat("K-Fold (LDA) -- AUC:", round(kf_lda$results$ROC, 4), "\n")
```

```
## K-Fold (LDA) -- AUC: 0.9486
```



```
cat("K-Fold (LDA) -- Sens:", round(kf_lda$results$Sens, 4), "\n")
```

```
## K-Fold (LDA) -- Sens: 0.998
```

```
cat("K-Fold (LDA) -- Spec:", round(kf_lda$results$Spec, 4), "\n")
```

```
## K-Fold (LDA) -- Spec: 0.2286
```

```
cat("K-Fold (LDA) -- LogLoss:", round(kf_lda$results$LogLoss, 4), "\n")
```

```
## K-Fold (LDA) -- LogLoss: 0.1604
```

```
set.seed(123)
kf_qda <- train(default ~ balance + income,
  data      = Default,
  method    = "qda",
  trControl = ctrl_kf,
  metric    = "ROC")
```

```
cat("K-Fold (QDA) -- AUC:", round(kf_qda$results$ROC, 4), "\n")
```

```
## K-Fold (QDA) -- AUC: 0.9483
```

```
cat("K-Fold (QDA) -- Sens:", round(kf_qda$results$Sens, 4), "\n")
```

```
## K-Fold (QDA) -- Sens: 0.9969
```

```
cat("K-Fold (QDA) -- Spec:", round(kf_qda$results$Spec, 4), "\n")
```

```
## K-Fold (QDA) -- Spec: 0.2796
```

```
cat("K-Fold (QDA) -- LogLoss:", round(kf_qda$results$LogLoss, 4), "\n")
```

```
## K-Fold (QDA) -- LogLoss: 0.1604
```

```
best_rbf_kf <- kf_rbf$results[which.max(kf_rbf$results$ROC), ]
```

```
tab_kfold <- data.frame(
  Metodo      = c(paste0("k-NN (k=", best_knn$k, ")"),
    "SVM Lineal", "SVM Radial", "SVM Polinomial",
    "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
  AUC          = round(c(best_knn$ROC, best_lin$ROC,
    best_rbf_kf$ROC, best_poly$ROC,
    kf_logit$results$ROC, best_nb_kf$ROC,
    kf_lda$results$ROC, kf_qda$results$ROC), 4),
  Sensibilidad = round(c(best_knn$Sens, best_lin$Sens,
    best_rbf_kf$Sens, best_poly$Sens,
    kf_logit$results$Sens, best_nb_kf$Sens,
```

```

        kf_lda$results$Sens, kf_qda$results$Sens), 4),
Especificidad = round(c(best_knn$Spec, best_lin$Spec,
        best_rbf_kf$Spec, best_poly$Spec,
        kf_logit$results$Spec, best_nb_kf$Spec,
        kf_lda$results$Spec, kf_qda$results$Spec), 4),
LogLoss      = round(c(best_knn$LogLoss, best_lin$LogLoss,
        best_rbf_kf$LogLoss, best_poly$LogLoss,
        kf_logit$results$LogLoss, best_nb_kf$LogLoss,
        kf_lda$results$LogLoss, kf_qda$results$LogLoss), 4)
)

# K-Fold (K=10) por clasificador :
tab_kfold

```

```

##          Metodo    AUC Sensibilidad Especificidad LogLoss
## 1    k-NN (k=30) 0.9034      0.9966      0.2737  0.3805
## 2      SVM Lineal 0.8170      1.0000      0.0152  0.2680
## 3      SVM Radial 0.6843      0.9972      0.2496  0.2293
## 4 SVM Polinomial 0.8944      0.9976      0.2501  0.1941
## 5 Reg. Logistica 0.9486      0.9961      0.3247  0.1585
## 6   Naive Bayes 0.9466      0.9961      0.2708  0.1632
## 7          LDA 0.9486      0.9980      0.2286  0.1604
## 8          QDA 0.9483      0.9969      0.2796  0.1604

```

```

kable(tab_kfold, row.names = FALSE, booktabs = TRUE,
      caption = "Validacion Cruzada K-Fold (K = 10) -- promedios por clasificador")

```

Cuadro 14: Validacion Cruzada K-Fold (K = 10) – promedios por clasificador

Metodo	AUC	Sensibilidad	Especificidad	LogLoss
k-NN (k=30)	0.9034	0.9966	0.2737	0.3805
SVM Lineal	0.8170	1.0000	0.0152	0.2680
SVM Radial	0.6843	0.9972	0.2496	0.2293
SVM Polinomial	0.8944	0.9976	0.2501	0.1941
Reg. Logistica	0.9486	0.9961	0.3247	0.1585
Naive Bayes	0.9466	0.9961	0.2708	0.1632
LDA	0.9486	0.9980	0.2286	0.1604
QDA	0.9483	0.9969	0.2796	0.1604

5.3. Bootstrap 0.632

El estimador Bootstrap 0.632 corrige el sesgo del bootstrap estandar ponderando el error aparente con el error out-of-bag:

$$\widehat{\text{Err}}^{(0,632)} = 0,368 \cdot \overline{\text{err}} + 0,632 \cdot \widehat{\text{Err}}^{(1)}$$

```

ctrl_boot632 <- trainControl(
  method      = "boot632",
  number      = 200,

```

```

classProbs      = TRUE,
summaryFunction = custom_summary,
savePredictions = "final"
)

```

```

set.seed(42)
bt_knn <- train(default ~ balance_sc + income_sc,
               data      = Default_sc,
               method    = "knn",
               trControl = ctrl_boot632,
               metric     = "ROC",
               tuneGrid  = data.frame(k = c(5, 10, 15, 20, 30)))

best_bt_knn <- bt_knn$results[which.max(bt_knn$results$ROC), ]
cat("Boot632 (k-NN) -- AUC:",      round(best_bt_knn$ROC,      4), "\n")

```

```
## Boot632 (k-NN) -- AUC: 0.9126
```

```
cat("Boot632 (k-NN) -- Sens:",      round(best_bt_knn$Sens,      4), "\n")
```

```
## Boot632 (k-NN) -- Sens: 0.9961
```

```
cat("Boot632 (k-NN) -- Spec:",      round(best_bt_knn$Spec,      4), "\n")
```

```
## Boot632 (k-NN) -- Spec: 0.2953
```

```
cat("Boot632 (k-NN) -- LogLoss:", round(best_bt_knn$LogLoss, 4), "\n")
```

```
## Boot632 (k-NN) -- LogLoss: 0.3489
```

```

set.seed(42)
bt_lin <- train(default ~ balance + income,
               data      = Default,
               method    = "svmLinear",
               trControl = ctrl_boot632,
               metric     = "ROC",
               preProcess = c("center", "scale"),
               tuneGrid  = data.frame(C = c(0.01, 0.1, 1, 5)))

```

```
## maximum number of iterations reached 0.008004536 0.007112494maximum number of iterations reached 0.0
```

```
## maximum number of iterations reached 0.00694649 0.006107843maximum number of iterations reached 0.00
```

```
## maximum number of iterations reached 0.0005371387 0.0005257411maximum number of iterations reached 0
```

```
## maximum number of iterations reached 0.007128143 0.006367638maximum number of iterations reached 0.0
```

```
## maximum number of iterations reached 0.006125715 0.006092909maximum number of iterations reached 0.0
```

```
## maximum number of iterations reached 0.0164702 0.01403763
```

```
best_bt_lin <- bt_lin$results[which.max(bt_lin$results$ROC), ]
cat("Boot632 (SVM Lineal) -- AUC:", round(best_bt_lin$ROC, 4), "\n")
```

```
## Boot632 (SVM Lineal) -- AUC: 0.8129
```

```
cat("Boot632 (SVM Lineal) -- Sens:", round(best_bt_lin$Sens, 4), "\n")
```

```
## Boot632 (SVM Lineal) -- Sens: 0.9999
```

```
cat("Boot632 (SVM Lineal) -- Spec:", round(best_bt_lin$Spec, 4), "\n")
```

```
## Boot632 (SVM Lineal) -- Spec: 0.0055
```

```
cat("Boot632 (SVM Lineal) -- LogLoss:", round(best_bt_lin$LogLoss, 4), "\n")
```

```
## Boot632 (SVM Lineal) -- LogLoss: 0.2878
```

```
set.seed(42)
bt_rbf <- train(default ~ balance + income,
  data      = Default,
  method    = "svmRadial",
  trControl = ctrl_boot632,
  metric    = "ROC",
  preProcess = c("center", "scale"))
```

```
## line search fails -4.064138 -0.2007948 1.005213e-05 -9.588554e-06 -6.072388e-08 -9.721092e-09 -5.171
```

```
## line search fails -3.872549 -0.06009329 1.186216e-05 -1.161035e-06 -4.052006e-07 -3.554344e-07 -4.39
```

```
## line search fails -3.433224 0.437012 1.024193e-05 -5.898686e-06 -1.490274e-07 -1.022387e-07 -9.23254
```

```
best_bt_rbf <- bt_rbf$results[which.max(bt_rbf$results$ROC), ]
cat("Boot632 (SVM Radial) -- AUC:", round(best_bt_rbf$ROC, 4), "\n")
```

```
## Boot632 (SVM Radial) -- AUC: 0.6927
```

```
cat("Boot632 (SVM Radial) -- Sens:", round(best_bt_rbf$Sens, 4), "\n")
```

```
## Boot632 (SVM Radial) -- Sens: 0.9974
```

```
cat("Boot632 (SVM Radial) -- Spec:", round(best_bt_rbf$Spec, 4), "\n")
```

```
## Boot632 (SVM Radial) -- Spec: 0.2501
```

```
cat("Boot632 (SVM Radial) -- LogLoss:", round(best_bt_rbf$LogLoss, 4), "\n")
```

```
## Boot632 (SVM Radial) -- LogLoss: 0.2282
```

```
set.seed(42)
bt_poly <- train(default ~ balance + income,
  data      = Default,
  method    = "svmPoly",
  trControl = ctrl_boot632,
  metric    = "ROC",
  preProcess = c("center", "scale"),
  tuneGrid  = expand.grid(degree = c(2,3), scale = 1,
    C = c(0.1, 1)))
```

```
## maximum number of iterations reached 0.0195031 0.01565662maximum number of iterations reached 0.0074
```

```
## maximum number of iterations reached 0.02409302 0.01865278maximum number of iterations reached 0.021
```

```
## maximum number of iterations reached 0.01773301 0.01588917maximum number of iterations reached 0.024
```

```
## line search fails -3.519467 -0.6025777 2.106711e-05 -1.492752e-05 -1.93618e-07 -1.376037e-07 -2.0248
```

```
## line search fails -3.977923 -1.189077 1.343857e-05 -8.444329e-06 -2.117768e-07 -1.822361e-07 -1.3071
```

```
## maximum number of iterations reached 0.02881367 0.02260435maximum number of iterations reached 0.023
```

```
## line search fails -3.928139 -0.958822 4.436817e-05 -3.23812e-05 -5.266084e-07 -3.930537e-07 -1.06371
```

```
## maximum number of iterations reached 6.520203e-05 6.507777e-05maximum number of iterations reached 0
```

```
## maximum number of iterations reached 0.02793606 0.02171123maximum number of iterations reached 0.011
```

```
best_bt_poly <- bt_poly$results[which.max(bt_poly$results$ROC), ]
cat("Boot632 (SVM Poly) -- AUC:", round(best_bt_poly$ROC, 4), "\n")
```

```
## Boot632 (SVM Poly) -- AUC: 0.8933
```

```
cat("Boot632 (SVM Poly) -- Sens:", round(best_bt_poly$Sens, 4), "\n")
```

```
## Boot632 (SVM Poly) -- Sens: 0.9977
```

```
cat("Boot632 (SVM Poly) -- Spec:", round(best_bt_poly$Spec, 4), "\n")
```

```
## Boot632 (SVM Poly) -- Spec: 0.2428
```

```
cat("Boot632 (SVM Poly) -- LogLoss:", round(best_bt_poly$LogLoss, 4), "\n")
```

```
## Boot632 (SVM Poly) -- LogLoss: 0.1972
```

```
set.seed(42)
bt_logit <- train(default ~ balance + income,
                  data      = Default,
                  method    = "glm",
                  family     = binomial(),
                  trControl = ctrl_boot632,
                  metric     = "ROC")
```

```
cat("Boot632 (Reg. Logistica) -- AUC:", round(bt_logit$results$ROC, 4), "\n")
```

```
## Boot632 (Reg. Logistica) -- AUC: 0.9487
```

```
cat("Boot632 (Reg. Logistica) -- Sens:", round(bt_logit$results$Sens, 4), "\n")
```

```
## Boot632 (Reg. Logistica) -- Sens: 0.996
```

```
cat("Boot632 (Reg. Logistica) -- Spec:", round(bt_logit$results$Spec, 4), "\n")
```

```
## Boot632 (Reg. Logistica) -- Spec: 0.3207
```

```
cat("Boot632 (Reg. Logistica) -- LogLoss:", round(bt_logit$results$LogLoss, 4), "\n")
```

```
## Boot632 (Reg. Logistica) -- LogLoss: 0.1588
```

```
set.seed(42)
bt_nb <- train(default ~ balance + income,
               data      = Default,
               method    = "naive_bayes",
               trControl = ctrl_boot632,
               metric     = "ROC")
```

```
best_bt_nb <- bt_nb$results[which.max(bt_nb$results$ROC), ]
cat("Boot632 (Naive Bayes) -- AUC:", round(best_bt_nb$ROC, 4), "\n")
```

```
## Boot632 (Naive Bayes) -- AUC: 0.9465
```

```
cat("Boot632 (Naive Bayes) -- Sens:", round(best_bt_nb$Sens, 4), "\n")
```

```
## Boot632 (Naive Bayes) -- Sens: 0.996
```

```
cat("Boot632 (Naive Bayes) -- Spec:", round(best_bt_nb$Spec, 4), "\n")
```

```
## Boot632 (Naive Bayes) -- Spec: 0.2654
```

```
cat("Boot632 (Naive Bayes) -- LogLoss:", round(best_bt_nb$LogLoss, 4), "\n")
```

```
## Boot632 (Naive Bayes) -- LogLoss: 0.1635
```

```
set.seed(42)
bt_lda <- train(default ~ balance + income,
  data      = Default,
  method    = "lda",
  trControl = ctrl_boot632,
  metric    = "ROC")
```

```
cat("Boot632 (LDA) -- AUC:", round(bt_lda$results$ROC, 4), "\n")
```

```
## Boot632 (LDA) -- AUC: 0.9487
```

```
cat("Boot632 (LDA) -- Sens:", round(bt_lda$results$Sens, 4), "\n")
```

```
## Boot632 (LDA) -- Sens: 0.9978
```

```
cat("Boot632 (LDA) -- Spec:", round(bt_lda$results$Spec, 4), "\n")
```

```
## Boot632 (LDA) -- Spec: 0.2342
```

```
cat("Boot632 (LDA) -- LogLoss:", round(bt_lda$results$LogLoss, 4), "\n")
```

```
## Boot632 (LDA) -- LogLoss: 0.1607
```

```
set.seed(42)
bt_qda <- train(default ~ balance + income,
  data      = Default,
  method    = "qda",
  trControl = ctrl_boot632,
  metric    = "ROC")
```

```
cat("Boot632 (QDA) -- AUC:", round(bt_qda$results$ROC, 4), "\n")
```

```
## Boot632 (QDA) -- AUC: 0.9485
```

```
cat("Boot632 (QDA) -- Sens:", round(bt_qda$results$Sens, 4), "\n")
```

```
## Boot632 (QDA) -- Sens: 0.997
```

```
cat("Boot632 (QDA) -- Spec:", round(bt_qda$results$Spec, 4), "\n")
```

```
## Boot632 (QDA) -- Spec: 0.2718
```

```
cat("Boot632 (QDA) -- LogLoss:", round(bt_qda$results$LogLoss, 4), "\n")
```

```
## Boot632 (QDA) -- LogLoss: 0.1606
```

```
tab_boot632 <- data.frame(
  Metodo      = c(paste0("k-NN (k=", best_bt_knn$k, ")"),
                  "SVM Lineal", "SVM Radial", "SVM Polinomial",
                  "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
  AUC          = round(c(best_bt_knn$ROC, best_bt_lin$ROC,
                        best_bt_rbf$ROC, best_bt_poly$ROC,
                        bt_logit$results$ROC, best_bt_nb$ROC,
                        bt_lda$results$ROC, bt_qda$results$ROC), 4),
  Sensibilidad = round(c(best_bt_knn$Sens, best_bt_lin$Sens,
                        best_bt_rbf$Sens, best_bt_poly$Sens,
                        bt_logit$results$Sens, best_bt_nb$Sens,
                        bt_lda$results$Sens, bt_qda$results$Sens), 4),
  Especificidad = round(c(best_bt_knn$Spec, best_bt_lin$Spec,
                        best_bt_rbf$Spec, best_bt_poly$Spec,
                        bt_logit$results$Spec, best_bt_nb$Spec,
                        bt_lda$results$Spec, bt_qda$results$Spec), 4),
  LogLoss      = round(c(best_bt_knn$LogLoss, best_bt_lin$LogLoss,
                        best_bt_rbf$LogLoss, best_bt_poly$LogLoss,
                        bt_logit$results$LogLoss, best_bt_nb$LogLoss,
                        bt_lda$results$LogLoss, bt_qda$results$LogLoss), 4)
)
```

```
# Bootstrap 0.632 (B=200) por clasificador :
tab_boot632
```

```
##      Metodo      AUC Sensibilidad Especificidad LogLoss
## 1  k-NN (k=30) 0.9126      0.9961      0.2953  0.3489
## 2  SVM Lineal 0.8129      0.9999      0.0055  0.2878
## 3  SVM Radial 0.6927      0.9974      0.2501  0.2282
## 4  SVM Polinomial 0.8933      0.9977      0.2428  0.1972
## 5  Reg. Logistica 0.9487      0.9960      0.3207  0.1588
## 6  Naive Bayes 0.9465      0.9960      0.2654  0.1635
## 7  LDA 0.9487      0.9978      0.2342  0.1607
## 8  QDA 0.9485      0.9970      0.2718  0.1606
```

```
kable(tab_boot632, row.names = FALSE, booktabs = TRUE,
      caption = "Bootstrap 0.632 (B = 200) -- promedios por clasificador")
```

Cuadro 15: Bootstrap 0.632 (B = 200) – promedios por clasificador

Metodo	AUC	Sensibilidad	Especificidad	LogLoss
k-NN (k=30)	0.9126	0.9961	0.2953	0.3489
SVM Lineal	0.8129	0.9999	0.0055	0.2878
SVM Radial	0.6927	0.9974	0.2501	0.2282
SVM Polinomial	0.8933	0.9977	0.2428	0.1972
Reg. Logistica	0.9487	0.9960	0.3207	0.1588

Metodo	AUC	Sensibilidad	Especificidad	LogLoss
Naive Bayes	0.9465	0.9960	0.2654	0.1635
LDA	0.9487	0.9978	0.2342	0.1607
QDA	0.9485	0.9970	0.2718	0.1606

6. Tabla Resumen Comparativa

Se consolida el AUC y el Log-Loss estimados por cada metodo de validacion cruzada, facilitando la comparacion final entre los ocho clasificadores: los cuatro de la Clase 7 (k-NN, SVM Lineal, SVM Radial, SVM Polinomial) y los cuatro de la Clase 5 (Regresion Logistica, Naive Bayes, LDA, QDA).

```
tab_auc_cv <- data.frame(
  Metodo = c(paste0("k-NN (k=", mod_knn$bestTune$k, ")"),
             "SVM Lineal", "SVM Radial", "SVM Polinomial",
             "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
  Muestra.Completa = round(c(pROC::auc(roc_knn), pROC::auc(roc_lin),
                              pROC::auc(roc_rbf), pROC::auc(roc_poly),
                              pROC::auc(roc_logit), pROC::auc(roc_nb),
                              pROC::auc(roc_lda), pROC::auc(roc_qda)), 4),
  Hold.Out.30 = tab_holdout$AUC,
  K.Fold.10 = tab_kfold$AUC,
  Boot.632 = tab_boot632$AUC
)

# AUC por clasificador y metodo de validacion :
tab_auc_cv
```

```
##          Metodo Muestra.Completa Hold.Out.30 K.Fold.10 Boot.632
## 1    k-NN (k=30)          0.9600      0.8961      0.9034      0.9126
## 2      SVM Lineal          0.5750      0.9471      0.8170      0.8129
## 3      SVM Radial          0.7044      0.6926      0.6843      0.6927
## 4 SVM Polinomial          0.8941      0.8951      0.8944      0.8933
## 5 Reg. Logistica          0.9491      0.9473      0.9486      0.9487
## 6    Naive Bayes          0.9468      0.9455      0.9466      0.9465
## 7              LDA          0.9491      0.9472      0.9486      0.9487
## 8              QDA          0.9489      0.9471      0.9483      0.9485
```

```
kable(tab_auc_cv, row.names = FALSE, booktabs = TRUE,
      caption = "AUC por clasificador y metodo de validacion")
```

Cuadro 16: AUC por clasificador y metodo de validacion

Metodo	Muestra.Completa	Hold.Out.30	K.Fold.10	Boot.632
k-NN (k=30)	0.9600	0.8961	0.9034	0.9126
SVM Lineal	0.5750	0.9471	0.8170	0.8129
SVM Radial	0.7044	0.6926	0.6843	0.6927
SVM Polinomial	0.8941	0.8951	0.8944	0.8933
Reg. Logistica	0.9491	0.9473	0.9486	0.9487
Naive Bayes	0.9468	0.9455	0.9466	0.9465
LDA	0.9491	0.9472	0.9486	0.9487
QDA	0.9489	0.9471	0.9483	0.9485

```
tab_ll_cv <- data.frame(
  Metodo = c(paste0("k-NN (k=", mod_knn$bestTune$k, ")"),
             "SVM Lineal", "SVM Radial", "SVM Polinomial",
             "Reg. Logistica", "Naive Bayes", "LDA", "QDA"),
```

```
Muestra.Completa = tab_logloss$LogLoss,
Hold.Out.30      = tab_holdout$LogLoss,
K.Fold.10        = tab_kfold$LogLoss,
Boot.632         = tab_boot632$LogLoss
)
```

```
# Log-Loss por clasificador y metodo de validacion :
tab_ll_cv
```

```
##          Metodo Muestra.Completa Hold.Out.30 K.Fold.10 Boot.632
## 1    k-NN (k=30)      0.1453      0.4244      0.3805      0.3489
## 2      SVM Lineal      0.3670      0.2898      0.2680      0.2878
## 3      SVM Radial      0.2244      0.2380      0.2293      0.2282
## 4 SVM Polinomial      0.1935      0.2051      0.1941      0.1972
## 5 Reg. Logistica      0.1579      0.1603      0.1585      0.1588
## 6    Naive Bayes      0.1626      0.1637      0.1632      0.1635
## 7          LDA      0.1600      0.1618      0.1604      0.1607
## 8          QDA      0.1595      0.1617      0.1604      0.1606
```

```
kable(tab_ll_cv, row.names = FALSE, booktabs = TRUE,
      caption = "Log-Loss por clasificador y metodo de validacion")
```

Cuadro 17: Log-Loss por clasificador y metodo de validacion

Metodo	Muestra.Completa	Hold.Out.30	K.Fold.10	Boot.632
k-NN (k=30)	0.1453	0.4244	0.3805	0.3489
SVM Lineal	0.3670	0.2898	0.2680	0.2878
SVM Radial	0.2244	0.2380	0.2293	0.2282
SVM Polinomial	0.1935	0.2051	0.1941	0.1972
Reg. Logistica	0.1579	0.1603	0.1585	0.1588
Naive Bayes	0.1626	0.1637	0.1632	0.1635
LDA	0.1600	0.1618	0.1604	0.1607
QDA	0.1595	0.1617	0.1604	0.1606

Los resultados permiten observar:

1. El **SVM Radial** suele obtener el mejor AUC en este conjunto, capturando la relacion no lineal entre **balance** y la probabilidad de impago de forma mas flexible que el SVM lineal.
2. El **k-NN** es sensible a la escala y al numero de vecinos k ; su AUC puede competir con los SVM cuando k se selecciona adecuadamente por validacion cruzada.
3. El **SVM Lineal** y el **SVM Polinomial** tienen un desempeno intermedio. El primero es mas interpretable; el segundo gana flexibilidad al modelar interacciones de orden superior.
4. Las estimaciones de muestra completa son optimistas. Los metodos de validacion cruzada producen estimaciones mas realistas del error fuera de muestra.
5. El desbalance severo de **Default** (3.3 % de impagos) explica la alta especificidad y baja sensibilidad de todos los modelos con umbral 0.5; vease la Seccion 7 para el manejo del desbalance.

6. Frente a los metodos lineales/parametricos de la Clase 5 (**Regresion Logistica** y **LDA**), los clasificadores no parametricos de la Clase 7 (**SVM Radial**, **k-NN**) solo aportan una ganancia relevante si la frontera de decision real no es aproximadamente lineal en **balance** e **income**; en **Default** esa ganancia suele ser modesta.
7. **Naive Bayes** y **QDA** permiten comparar el costo de sus supuestos (independencia condicional y covarianzas distintas por grupo, respectivamente) frente a la mayor flexibilidad, pero menor interpretabilidad, de los SVM.

7. Clases Desbalanceadas con SVM Radial

Se aplican las técnicas de balanceo al mejor modelo (SVM Radial) para evaluar su impacto en la sensibilidad, usando el mismo Hold-Out 70/30.

```
set.seed(123)
n_def <- nrow(Default)
idx <- sample(1:n_def, floor(0.7 * n_def))
train_b <- Default[idx, ]
test_b <- Default[-idx, ]

y_test_b <- test_b$default
y_test_b_bin <- test_b$default_bin

ajustar_svm_rbf <- function(sampling_method = NULL, train_data, label) {
  ctrl <- trainControl(method = "cv",
                        number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary,
                        sampling = sampling_method)

  # kernlab calibra las probabilidades de svmRadial con un submodelo
  # (Platt scaling) que puede fallar por su aleatoriedad interna y devolver
  # un vector de probabilidades incompleto o con NA; se reintenta con otra
  # semilla si eso ocurre
  semillas <- c(123, 456, 789, 2024)
  pi_hat <- NULL
  for (s in semillas) {
    set.seed(s)
    mod <- train(default ~ balance + income,
                  data = train_data,
                  method = "svmRadial",
                  trControl = ctrl,
                  metric = "ROC",
                  preProcess = c("center", "scale"))
    intento <- predict(mod, newdata = test_b, type = "prob")[, "Yes"]
    if (length(intento) == nrow(test_b) && !anyNA(intento)) {
      pi_hat <- intento
      break
    }
  }
  if (is.null(pi_hat)) {
    stop("No se pudo calibrar las probabilidades de SVM Radial (", label, ")")
  }

  y_pred <- clf(pi_hat)
  cm <- confusionMatrix(y_pred, y_test_b, positive = "Yes")
  VP <- cm$table[2,2]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  data.frame(
    Tecnica = label,
    Accuracy = round(cm$overall["Accuracy"], 4),
    Sensibilidad = round(cm$byClass["Sensitivity"], 4),
    Especificidad = round(cm$byClass["Specificity"], 4),
```

```

F1      = round(2*VP/(2*VP+FP+FN),      4),
AUC      = round(as.numeric(
      pROC::auc(pROC::roc(y_test_b_bin, pi_hat,
      quiet=TRUE))), 4)
)
}

```

```

res_sin      <- ajustar_svm_rbf(NULL,      train_b, "Sin balanceo (p=0.5)")
res_under    <- ajustar_svm_rbf("down",    train_b, "Undersampling")
res_over     <- ajustar_svm_rbf("up",      train_b, "Oversampling")

```

```
## line search fails -2.161672 0.1842652 6.525823e-05 -5.099774e-05 -7.109168e-08 5.974632e-08 -7.68624e-08
```

```
## line search fails -2.26042 0.2069185 0.0001106733 -9.002125e-05 -1.718955e-07 1.391911e-07 -3.15544e-07
```

```
## line search fails -2.251186 0.2577248 1.293372e-05 -1.099075e-05 -1.985328e-08 1.649651e-08 -4.38085e-08
```

```
## line search fails -2.282678 0.2453938 4.741111e-05 -3.982639e-05 -5.885873e-08 4.857114e-08 -4.72497e-08
```

```
## line search fails -2.231809 0.2518753 2.751347e-05 -2.303802e-05 -4.180973e-08 3.445518e-08 -1.94411e-08
```

```
## line search fails -2.213855 0.2224365 4.508377e-05 -3.643778e-05 -6.606739e-08 5.455378e-08 -4.96638e-08
```

```
## line search fails -2.121851 0.1714752 1.200455e-05 -9.071549e-06 -1.585942e-08 1.295135e-08 -3.07874e-08
```

```
## line search fails -2.110943 0.1607371 1.43794e-05 -1.098581e-05 -1.823672e-08 1.535672e-08 -4.30939e-08
```

```
res_smote    <- ajustar_svm_rbf("smote", train_b, "SMOTE")
```

```

# Adicionalmente: corte optimo sobre modelo sin balanceo
mod_base_b <- train(default ~ balance + income,
      data      = train_b,
      method    = "svmRadial",
      trControl = trainControl(method = "cv", number = 5,
      classProbs = TRUE,
      summaryFunction = twoClassSummary),
      metric     = "ROC",
      preProcess = c("center","scale"))

pi_base_b <- predict(mod_base_b, newdata = test_b, type = "prob")[,"Yes"]

cortes_b <- seq(0.01, 0.50, by = 0.01)
dif_b <- sapply(cortes_b, function(c){
  pred_c <- factor(ifelse(pi_base_b >= c, "Yes","No"), levels = c("No","Yes"))
  cm_c    <- confusionMatrix(pred_c, y_test_b, positive = "Yes")
  abs(cm_c$byClass["Sensitivity"] - cm_c$byClass["Specificity"])
})
corte_b_opt <- cortes_b[which.min(dif_b)]

y_pred_opt <- factor(ifelse(pi_base_b >= corte_b_opt, "Yes","No"),

```

```

        levels = c("No", "Yes"))
cm_opt_b <- confusionMatrix(y_pred_opt, y_test_b, positive = "Yes")
VP <- cm_opt_b$table[2,2]; FP <- cm_opt_b$table[2,1]; FN <- cm_opt_b$table[1,2]

res_corte <- data.frame(
  Tecnica      = paste0("Corte optimo (p=", corte_b_opt, ")"),
  Accuracy     = round(cm_opt_b$overall["Accuracy"], 4),
  Sensibilidad = round(cm_opt_b$byClass["Sensitivity"], 4),
  Especificidad = round(cm_opt_b$byClass["Specificity"], 4),
  F1           = round((2*VP)/(2*VP+FP+FN), 4),
  AUC          = round(as.numeric(
    pROC::auc(pROC::roc(y_test_b_bin, pi_base_b,
      quiet=TRUE))), 4)
)

tab_balanceo <- bind_rows(res_sin, res_under, res_over, res_smote, res_corte)

# Comparacion de tecnicas de balanceo - SVM Radial :
tab_balanceo

##              Tecnica Accuracy Sensibilidad Especificidad      F1
## Accuracy...1 Sin balanceo (p=0.5)  0.9720          0.24      0.9972 0.3636
## Accuracy...2 Undersampling      0.8503          0.87      0.8497 0.2793
## Accuracy...3 Oversampling       0.8560          0.85      0.8562 0.2824
## Accuracy...4 SMOTE              0.8723          0.85      0.8731 0.3074
## Accuracy...5 Corte optimo (p=0.03) 0.9713          0.51      0.9872 0.5426
##              AUC
## Accuracy...1 0.6926
## Accuracy...2 0.9201
## Accuracy...3 0.9098
## Accuracy...4 0.8921
## Accuracy...5 0.6912

kable(tab_balanceo, row.names = FALSE, booktabs = TRUE,
      caption = "Comparacion de tecnicas de balanceo -- SVM Radial sobre Default (test 30\\%)")

```

Cuadro 18: Comparacion de tecnicas de balanceo – SVM Radial sobre Default (test 30 %)

Tecnica	Accuracy	Sensibilidad	Especificidad	F1	AUC
Sin balanceo (p=0.5)	0.9720	0.24	0.9972	0.3636	0.6926
Undersampling	0.8503	0.87	0.8497	0.2793	0.9201
Oversampling	0.8560	0.85	0.8562	0.2824	0.9098
SMOTE	0.8723	0.85	0.8731	0.3074	0.8921
Corte optimo (p=0.03)	0.9713	0.51	0.9872	0.5426	0.6912

Los resultados permiten observar:

1. Sin balanceo, la sensibilidad es muy baja: el modelo predice casi todo como “No”, optimizando la accuracy pero fallando en detectar impagos.
2. El **Undersampling** y **Oversampling** mejoran la sensibilidad substancialmente al reequilibrar las proporciones de clase durante el entrenamiento.

3. **SMOTE** suele lograr el mejor equilibrio entre sensibilidad y especificidad al generar ejemplos sintéticos informativos de la clase minoritaria, en lugar de simplemente replicar observaciones existentes.
4. El **corte óptimo** es una alternativa eficaz cuando no se desea modificar los datos de entrenamiento: ajusta el umbral de decisión para equilibrar sensibilidad y especificidad sin alterar el modelo ajustado.
5. El AUC es idéntico para el modelo sin balanceo y con corte óptimo (mismo modelo, distinto umbral), mientras que los métodos de balanceo pueden cambiar el AUC al modificar el modelo entrenado.

8. Ejercicio Propuesto

Ejercicio Considere la base de datos **Weekly** de ISLR2, que contiene 1089 retornos semanales del mercado bursatil americano entre 1990 y 2010. La variable respuesta es **Direction** (Up / Down) en funcion de los retornos rezagados **Lag1**, **Lag2**, **Lag3**, **Lag4**, **Lag5** y del volumen **Volume**.

Usando como conjunto de entrenamiento los años 1990–2008 y como conjunto de prueba los años 2009–2010:

- Ajuste los cuatro clasificadores (k-NN con k optimo, SVM Lineal, SVM Radial y SVM Polinomial) usando los predictores **Lag1**, **Lag2** y **Volume**. Para cada modelo reporte la matriz de confusion y calcule Accuracy, Sensibilidad, Especificidad, F1 Score y AUC.
- Grafique las cuatro curvas ROC en un mismo panel e identifique visualmente cual clasificador tiene mejor capacidad discriminante.
- Estime el error de prediccion de los cuatro modelos mediante validacion cruzada K-Fold con $K = 10$ sobre el conjunto de entrenamiento. Reporte AUC y Log-Loss. Compare con el Hold-Out del literal (a).
- Repita la estimacion del literal (c) utilizando Bootstrap 0.632 con $B = 200$ muestras. ¿Las conclusiones cambian respecto al K-Fold?
- Verifique si existe desbalance en **Direction**. Aplique **SMOTE** y **undersampling** al mejor clasificador identificado en (a) y compare las metricas (incluyendo F1 y AUC) con el modelo sin balancear.
- Elabore una tabla resumen comparativa (similar a la Seccion 6 de este material) e indique cual clasificador recomendaria para este problema y por que.

Referencias

- [Hastie et al.(2008)] Trevor Hastie, Robert Tibshirani and Jerome Friedman (2008) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics.
- [James et al.(2021)] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani (2021) *An Introduction to Statistical Learning with Applications in R*, 2nd edition. Springer Series in Statistics.